

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Content

Document Control	3
PRELIMINARY ROUND (Test Strategy & Planning)	3
1. Scope & Requirements Traceability	3
2. Risk Assessment & Mitigation Strategy	3
3. Test Environment & Execution Strategy	5
4. CI/CD Release Thresholds & Automation Gates	5
5. Test Case Specifications (Drafts)	6
6. Test Strategy & Plan Sign-Off	7

Document Control

Field	Detail
System Under Test (SUT)	ProcureIQ: AI-Powered Vendor and Procurement Agent
Team Repo URL	https://github.com/husn44/64PinkPixels
Project Board URL	https://trello.com/invite/b/69ece0e316fd2519ff5edd03/ATTI0dbec622865e857ea22145e79e20d02f69C6AF13/hackathon-2026-pinkpixels
Live Deployment URL (optional for preliminary round)	N/A

Objective:

The primary objective is to ensure that **ProcureIQ** can accurately extract financial data from unstructured PDFs, conduct real-time adversarial research for vendor "Red Flags," and generate legally formatted Purchase Orders (PO) through an agentic workflow powered by **ILMU-GLM-5.1**, maintaining extraction confidence scoring for data accuracy and high API responsiveness.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix**". So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.

1.1 In-Scope Core Features

- **Multi-Source PDF Extraction:** Text extraction via pdfplumber followed by Z.AI GLM to transform unstructured PDF quote text into structured JSON data (prices, dates, names).
- **Adversarial Research Engine:** Real-time web scraping to identify vendor "Red Flags" (reputation, financial stability, legal news).
- **Multi-Factor Analytical "Brain":** Applying a 4-factor logic (Current Quotes, News, Market Benchmarks, and Historical Internal Data) to generate recommendations.
- **Automated Fulfillment:** Generative creation of professional, legally formatted Purchase Orders (PO) and confirmation email drafts

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2 Out-of-Scope

- Manual data entry/spreadsheet transcription (automated by the system).
- Final Purchase Order signature/approval (requires explicit human review).

2. Risk Assessment & Mitigation Strategy

Calculated using the Risk Score = Likelihood (1-5) × Severity (1-5) matrix.

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Financial Hallucination: AI generates a realistic but mathematically wrong price.	3	5	15 (High)	Implement a confidence score (0.0 to 1.0) for each extraction with human-in-the-loop review in Step 2 of the dashboard.	Verify GLM extraction output against original PDF data, checking confidence scores and flagged low-confidence items.
Parsing Failure: Unable to read low-resolution or mobile photo quotes.	4	3	12 (High)	Restrict input to digital PDFs only; use pdfplumber for text extraction with a GLM retry on truncated text (shorter input fallback)	Test with various PDF layouts to ensure text extraction and GLM data normalization remains consistent.
Data Freshness: Missing a very recent legal event or "Red Flag".	2	4	8 (Medium)	Use asynchronous Web Research Flows to fetch live external news.	Execute adversarial search tests with known recent news items.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact

© UMHackathon 2026

Email: umhackathon@um.edu.my

4	<i>Likely</i>	4	<i>Major Impact</i>
5	<i>Almost Certain</i>	5	<i>Critical Failure of the system</i>

Risk Score = Likelihood × Severity

Risk Level Reference:

<i>Risk Score</i>	<i>Risk Level</i>	<i>Recommended Action</i>
<i>1 – 5</i>	<i>Low</i>	<i>Monitor only. Acceptable risks.</i>
<i>6 – 10</i>	<i>Medium</i>	<i>Mitigate + Testing</i>
<i>11 – 15</i>	<i>High</i>	<i>Must need mitigating and through testing is required</i>
<i>16 – 25</i>	<i>Critical</i>	<i>Priority is Highest. Need extensive level of testing.</i>

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
 - Scope: *models.py* validation and *po_generator.py* formatting.
 - Execution: *PyTest* framework.
 - Pass Condition: 100% success on *Pydantic* data model parsing.
- **Integration Test**
 - Scope: *main.py* (FastAPI) to *ai_service.py* (Z.AI API) connection.
 - Workflow: Ensure 9 REST endpoints communicate without 500 errors.
- **Test Environment (CI/CD Practice):**
 - Local: *Streamlit* frontend (*app.py*) on port 8501.
 - CI/CD: Local testing via *PyTest* before merging. *GitHub Actions* pipeline to be set up for automated test execution on Pull Requests.
- **Regression Testing & Pass/Fail Rules:**
 - Execution Phase: Regression testing is triggered automatically via *GitHub Actions* every time a feature branch is merged into *main*. It covers all 4 core modules: PDF Extraction, Adversarial Research, Multi-Factor Brain logic, and PO Generation.
 - Pass/Fail Condition: A test case is marked Passed only if the JSON output from *ai_service.py* perfectly matches the schema defined in *models.py* and the mathematical "Landed Cost" matches the expected value within a 0.01% variance. Any deviation is logged in the Defect Log and blocks the merge.
 - Continuation Rule: We follow a "Gatekeeper" approach. Baseline extraction tests (Phase 1) must achieve a 100% pass rate before the pipeline allows execution of the Research or Fulfillment tests.
- **Test Data Strategy:**
 - Manual Data: We have curated sample PDF quote templates (varying layouts, fonts, and table structures) to test the robustness of the *pdfplumber* text extraction and Z.AI GLM extraction engine.
 - Mock User Roles: The MVP has no authentication system. All users have full dashboard access. User role differentiation is identified as a future enhancement for production deployment.
 - Automated Data: Historical pricing data is stored in [JSONBin.io](https://jsonbin.io). The system retrieves all quote history via the *JSONBin* REST API to test the "Historical Comparison" feature of the Multi-Factor Brain.

- **Passing Rate Threshold:**

- *Global Threshold: A minimum of 90% of total test cases must pass for a successful deployment.*
- *Critical "Zero-Tolerance" Gate: * 100% Pass Rate required for all Phase 1 (Extraction) and Phase 5 (PO Generation) tests.*

Reason: Mathematical errors in procurement or legally binding document generation carry critical financial risk and cannot be bypassed.

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	Requirements	Project	Pass/Failed
Automatic Build	Zero Build Error	0 Errors	Passed
Unit Tests	100% Passing Rate	100%	Passed
Test Coverage	Minimum 85%	87%	Passed
Code Quality	Zero Linting Error	Clean (Black/Flake8)	Passed

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	93%	Passed
AI Output Pass Rate	Minimum of 80% of documented prompt	85%	Passed
Critical Bugs	Zero P0/01 Bugs	0	Passed
API Performance	Response Time < 800ms	120 s	Failed
Security	API keys hidden (.env)	Verified	Passed

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases **MUST include at least:**

- **1 Happy Case (Entire Flow):** This is the "Golden Path" where a user journey is tested end to end.
- **1 Specific Case (Negative/Edge Test):** This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.
- **2 Non-Functional (NFR) Tests (Performance/Load):** This test consists of non-feature-based tests but architectural tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case: End-to-End Procurement	Verify the "Golden Path" from PDF upload to PO generation.	1. Open ProcureIQ dashboard (localhost: 8501). 2. Upload 3 vendor PDFs (Phase 1).	System extracts data accurately, identifies risks, and generates a professional PDF PO within 30s.	Successfully extracted data from 3 PDFs. Red flags identified. PO generated in 24.5s. Status: Passed

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			3. Run "Adversarial Research" (Phase 2). 4. Compare results in "The Brain" (Phase 3). 5. Generate & Download PO (Phase 5).		
TC-02	Negative Case: Invalid File Format	Ensure the system handles non-PDF or corrupted files without crashing.	1. Go to Upload Screen. 2. Attempt to upload a .png or .exe file. 3. Click "Process."	Streamlit's file uploader restricts uploads to .pdf files only. None-PDF files cannot be selected in the file picker.	Upload was blocked, but the UI did not show a clear error message to the user. Status: Failed
TC-03	NFR (Performance): Extraction Latency	Verify Z.AI GLM extraction speed under normal load.	1. Upload a complex 5-page PDF quote. 2. Measure time from "Upload" to "JSON Structured View."	Total extraction and normalization time should be < 15 seconds.	Extraction took 19.2 seconds due to complex table structures. Status: Failed
TC-04	NFR (Load): Concurrent Processing	Verify system stability when multiple vendors are processed.	1. Use Postman to send 5 concurrent PDF processing requests to /extract .	All 5 requests complete successfully without 504 Timeout errors.	4/5 requests succeeded; 1 request timed out at 30s. Status: Failed

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. Prompt/Response Test Pairs

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	PDF extraction: Upload a vendor quote PDF containing 3 line items with prices, quantities and terms.	Valid JSON with vendor_name, 3 items with item_price, quantity, normalized_unit and confidence score above 0.7	Extracted 2 of 3 items correctly. Third item had low confidence (0.4) due to overlapping table text.	Partially Pass
AI-02	Adversarial analysis: Provide 2 vendor quotes, market benchmarks and reputation data for comparison.	Valid JSON with analyses for each vendor containing overall_score (0-10), price_vs_market, reputation_risk and a winner recommendation.	Both vendors analyzed with scores. Winner correctly identified but one reputation_risk labeled "unknown" due to no web results.	Fail
AI-03	Email drafting: Provide selected vendor with agreed price RM 2,500.00 and 14-day delivery.	Professional acceptance email referencing exact price RM 2,500.00 and delivery timeline.	Email included price correctly but mentioned "approximately 2 weeks" instead of "14 days" for delivery.	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	No hard rate limit; limited by GLM API rate limits and JSONBin.io write rate (1 write/sec)
Input used while testing	Multiple concurrent PDF uploads (3+ files simultaneously)
Expected Behavior	System processes files sequentially per upload request. JSONBin rate limit (1 write/sec) is handled with a 1-second delay between writes.
Actual Behavior	System handles multiple uploads within a single request. Large batches may experience slow response due to sequential GLM and JSONBin API calls.
Status	Passed

6.3. Adversarial/Edge Prompt Test

- Scenario: A user provides a "Malformed PDF" where text is overlapping.
- System Response: The Z.AI GLM identifies low confidence in the extraction.
- Handling: ProcureIQ assigns a low confidence score (0.0) and the item is flagged for manual review in the dashboard. The user can verify and proceed or re-upload a clearer PDF.

6.4. Hallucinating Handling

To mitigate AI hallucinations, the system uses a confidence scoring mechanism. Each extracted item receives a confidence score (0.0 to 1.0) from the GLM. Low-confidence extractions are flagged for manual review in Step 2 of the dashboard, where the user can verify all extracted data before proceeding to research and analysis. The GLM extraction prompt explicitly instructs the model to extract prices as-is from the PDF text rather than calculating or inferring values.

Important Note:

© UMHackathon 2026

Email: umhackathon@um.edu.my

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.